

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It identifies code intended to run when the file is executed directly
- B)It creates the main function
- C)It prevents all imports
- D)It prevents syntax errors

Q2. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A)-10
- B)30
- C)10
- D)20

Q3. What is the output?

```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)A
- B)A C
- C)B C
- D)B

Q4. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)First provides the module namespace; second imports sqrt directly
- B)First imports only sqrt
- C)Both import only sqrt
- D)There is no difference

Q5. What is the output?

```
def fun(x):
    x += 10
    return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

- A)5 5
- B)15 15
- C)5 15
- D)15 5

Q6. What will be the output?

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

```
fun()
```

- A)10
- B)NameError
- C)20
- D)UnboundLocalError

Q7. Which object keeps track of modules that have already been imported during the current Python process?

- A)sys.imports
- B)sys.modules
- C)__module__
- D)sys.path

Q8. What happens when this code runs?



```
def first():
    second()

def second():
    third()

def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)NameError
B)TypeError
C)Handled
D)RuntimeError cannot propagate through functions

Q9. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)20
B)None
C)NameError
D)10

Q10. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

A)Error
B)Only Hello

C)x
display
D)10
Hello

Q11. What is the output?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal x
        x = x + 5
        return x
```

```
    inner()
    return x
```

```
print(outer())
```

A)10
B)5
C)15
D)UnboundLocalError

Q12. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)Error
B>Hello None
C)None
D>Hello

Q13. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

A)A, followed by propagation of the same exception
B)B
C)A, and execution ends normally
D)No Output



Q14. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

- A)5
- B)Error
- C)func(5)
- D)10

Q15. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

- A)ZeroDivisionError occurs before Exception
- B)Exception is a superclass of ZeroDivisionError
- C)except blocks execute randomly
- D)ZeroDivisionError is not an exception

Q16. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

- A)A B
- B)Nothing
- C)A
- D)B

Q17. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)50

B)UnboundLocalError

C)10

D)NameError

Q18. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

- A)None
- B)NameError
- C)10
- D)15

Q19. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A)[1, 3, 5]
- B)[2, 4]
- C)[2, 4, 6, 8, 10]
- D)[4, 8]

Q20. What happens here?

```
def fun(a):  
    return a
```

```
print(fun())
```

- A)0
- B)NameError
- C)TypeError
- D)None

Q21. What will be printed?



try:

print(10 / 0)

except ZeroDivisionError:

print("A")

except Exception:

print("B")

A)B

B)Nothing

C)A B

D)A

Q22. What will be printed?

x = 10

def outer():

x = 20

def inner():

print(x)

x = 30

inner()

outer()

A)20

B)10

C)30

D)NameError

Q23. What will happen?

def outer():

x = 10

def inner():

nonlocal y

y = 20

inner()

outer()

A)20

B)10

C)SyntaxError

D)NameError

Q24. What happens?

def fun(a, b):

return a + b

print(fun(10, a=20))

A)20

B)TypeError

C)30

D)SyntaxError

Q25. What will normally happen?

import math

import math

A)The module is normally loaded once and reused

B)ImportError

C)The module is normally loaded and executed twice

D)SyntaxError

Q26. What will happen?

def fun(a, b=10):

return a + b

print(fun(b=5, a=20))

A)25

B)TypeError

C)SyntaxError

D)15

Q27. What is the output?

try:

print(10 / 0)

except Exception:

print("A")

except ZeroDivisionError:

print("B")

A)A

B)B

C)A B

D)SyntaxError

Q28. What is the output?

def fun():

try:

print("A")

raise ValueError

except ValueError:

print("B")

return 10

else:



```

    print("C")
    return 20
finally:
    print("D")

```

```
print(fun())
```

A)A

B

D

10

B)A

C

D

20

C)A

B

C

D

10

D)A

B

10

D

Q29. What happens when this function is called?

```

def fun(a=10, b):
    return a + b

```

A)It executes correctly

B)TypeError

C)SyntaxError

D)NameError

Q30. What happens here?

```

class MyError(Exception):
    pass

```

```

def test():
    raise MyError("Something went wrong")

```

```

try:
    test()
except MyError as e:
    print(type(e).__name__)

```

A)Exception

B)TypeError

C)MyError

D)Something went wrong

Q31. What will be printed?

```

def fun(a, b):
    return a + b

```

```
print(fun(10, b=20))
```

A)30

B)SyntaxError

C)TypeError

D)20

Q32. What will be printed?

```

def make_counter():
    count = 0

```

```

    def counter():
        nonlocal count
        count += 1
        return count

```

```
    return counter
```

```

a = make_counter()
b = make_counter()

```

```

print(a())
print(a())
print(b())
print(a())

```

A)0

1

0

2

B)1

1

1

1

C)1

2

3

4

D)1

2

1

3



Q33. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

print(fun(3))

- A)3
- B)6
- C)None
- D)5

Q34. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

print(result)

- A)0
- B)10
- C)9
- D)24

Q35. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

- A)Nothing
- B)e
- C)ValueError
- D)Wrong Value

Q36. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

- A)30
- 30
- B)30
- 20
- C)20
- 30
- D)20
- 10

Q37. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A)25
- B)5
- C)5.0
- D)sqrt(25)

Q38. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A>Error
- B)15
- C)10
- D)5

**Q39. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)B
- B)Nothing
- C)A
- D)A followed by B

Q40. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:
```



```

    raise ValueError("negative")
    return x
except ValueError:
    return 100
finally:
    print("cleanup")

```

print(fun(-5))

- A)cleanup
- 100
- B)cleanup
- 5
- C)negative
- cleanup
- D)100
- cleanup

Q41. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)Module loaded
- Module loaded
- 10
- B)10
- 10
- C)ImportError
- D)Module loaded
- 10

Q42. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)The local file can shadow the intended standard-library module

B)Python automatically renames the local file

C)random cannot be imported

D)Python will always reject it

Q43. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)NameError
- B)36
- C)6.0
- D)6

Q44. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
```

```
b = fun()
```

```
print(a)
```

```
print(b)
```

- A)[10]
- [10, 10]
- B)[10, 10]
- [10]
- C)[10]
- [10]
- D>Error

Q45. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)Finally only
- B)10 followed by Finally
- C)10 only
- D)Finally followed by 10

Q46. What will be printed?

```
try:
    x = int("10")
```



except ValueError:

print("A")

else:

print("B")

finally:

print("C")

A)A

B)B C

C)B

D)A C

Q47. What will be printed?

class InvalidAgeError(Exception):

pass

def check_age(age):

if age < 18:

raise InvalidAgeError("Age is too low")

return "Allowed"

try:

print(check_age(15))

except InvalidAgeError as e:

print(type(e).__name__)

print(e)

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

15

D)InvalidAgeError

Age is too low

Q48. What is the output?

def fun():

try:

return 10

finally:

return 20

print(fun())

A)RuntimeError

B)10

C)20

D)None

Q49. Suppose test.py contains:

print("A")

if __name__ == "__main__":

print("B")

If another Python program executes:

import test

what will normally be printed?

A)A and B

B)Nothing

C)B

D)A

Q50. What will happen?

def fun(n):

if n == 0:

return 1

return n * fun(n)

print(fun(3))

A)3

B)TypeError

C)RecursionError

D)6

